
Overview

The second genre chapter where movement *is* the game. Students meet the same platformer three times, each version a complete game: a single screen of static platforms and coins; a scrolling Tiled level with a hero that feels good to control, patrolling enemies and hazards; and a three-level game with moving and one-way platforms, ladders, swooping bats, checkpoints, lives, a HUD scene and a level select.

The Phaser ideas are **Arcade gravity and** `body.blocked.down`, **acceleration, drag and max velocity, tile properties** (`setCollisionByProperty` , `tile.properties`), `checkCollision` **and process callbacks** for one-way collisions, and **immovable bodies that carry riders**. The design ideas are **game feel** (variable jump, coyote time, jump buffering), **small state machines** (hero, bat), **interfaces** for enemies, **passing a function** instead of an object, and **data-driven levels**. Almost everything else is revision of Chapters 2-11, used for real.

Prerequisites

- Chapters 7-11 in particular: sprite sheet animations and the hero state machine (7), colliders, overlaps, groups and process callbacks (8), acceleration, drag and gravity (9), tilemaps and cameras (10), Tiled maps and object layers (11)
- Chapters 4 and 6 for the advanced version's HUD scene and registry events; Chapter 6 for `localStorage`
- Java interfaces (for `Enemy`) and, ideally, a one-method interface or lambda in Java (for `LadderFinder`)

Learning outcomes

Students can:

1. make a hero that runs and jumps with Arcade Physics, jumping only when `body.blocked.down`
2. explain and implement a variable jump height, coyote time and jump buffering, and tune movement constants by play-testing
3. build a level from a Tiled map using tile properties for collision and hazards, and place game objects from the Objects layer
4. decide between stomp and hurt in an overlap callback, and use a process callback to make tile hazards fair
5. make one-way platforms with `checkCollision`, and platforms that carry the hero
6. structure a multi-level game: one data-driven game scene, a HUD scene fed by the registry, checkpoints and lives, and progress saved between sessions

Suggested session plan (2 x 2-hour labs)

Session 1 - the simple and intermediate versions

Time	Activity
0:00 - 0:10	Play <code>ch15_platformer_simple</code> . Ask: "what does it feel like?" (stiff; every jump the same). Keep the answers for later
0:10 - 0:30	Slides 3-5: gravity, the Arcade Sprite and <code>declare body</code> , <code>blocked.down</code> ; then in the code, the animation state machine and collider vs overlap. Live-code the jump (see demos)
0:30 - 0:50	Challenges 1 and 2
0:50 - 1:05	Play <code>ch15_platformer_intermediate</code> . Compare with the list from 0:00
1:05 - 1:25	Slides 7-9: the Tiled map and its properties, game feel (acceleration and drag in the code, then the jump cut, coyote time, buffering). The coyote demo
1:25 - 1:45	Slides 10-11: slimes, stomp or hurt, fair hazards
1:45 - 2:00	Start challenge 4 (wall jump)

Session 2 - the advanced version

Time	Activity
0:00 - 0:10	Play <code>ch15_platformer_advanced</code> , all three levels (unlock them by editing <code>localStorage</code> in the dev tools, or play level 1 fast)
0:10 - 0:40	Slides 13-17: data-driven levels, one-way and moving platforms, ladders and <code>LadderFinder</code> , bats, the <code>Enemy</code> interface, checkpoints and the HUD
0:40 - 1:00	Finish challenge 4
1:00 - 2:00	Challenges 3, 5 and 6 - or a free "make your own level" exercise in Tiled

A single 3-hour lab can cover the simple and intermediate versions with challenges 1, 2 and 4, and leave the advanced version for reading at home.

Key points to stress

- `blocked.down` **is set by collisions**, and only by a collider or the world bounds. It is the question "am I standing on something?", asked of the physics engine rather than worked out from positions
- **a jump is a velocity; gravity does the rest.** The height is $\text{speed}^2 / (2 \times \text{gravity})$: students who want "a jump 3 tiles high" can work out the speed rather than guessing
- **game feel is numbers.** Acceleration, drag, jump speed, gravity, jump cut, coyote time and buffer are all constants at the top of `Hero.ts` . Encourage students to change them one at a time and play: this is exactly what game designers do
- **remember *when*, not just *whether*** - coyote time and buffering both come from storing a timestamp. The same idea gives invulnerability after respawning (`safeUntil`) and the wall-jump key lock in challenge 4
- **tiles say what they are.** No tile numbers in the code: properties set once in the tileset apply to every tile in every level
- **overlap with a tilemap layer includes empty tiles** - the process callback is not optional
- **the scene decides what a touch means**, not the hero or the enemy. That keeps `Slime` , `Bat` and `Hero` independent - and in the advanced version, the `Enemy` interface keeps the scene independent of which enemies exist
- **the data drives the levels.** Adding a level means a map and a line in `levels.ts`

Common problems and errors

What students see	Cause	Fix
<code>TS2531 [ERROR]:</code> Object is possibly 'null'. on every <code>this.body...</code> line, and <code>Property</code> 'acceleration' does not exist on type 'Body StaticBody'.	the <code>declare body:</code> <code>Phaser.Physics.Arcade.Body;</code> line is missing from a class extending <code>Phaser.Physics.Arcade.Sprite</code>	add it (and discuss why it is safe: the constructor gives the sprite a dynamic body)
<code>TS4114 [ERROR]:</code> This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	<code>preUpdate</code> without <code>override</code>	add <code>protected override</code>
The hero never animates, or is stuck on one frame	<code>super.preUpdate(time, delta)</code> missing from <code>preUpdate()</code> , or <code>play(key)</code> without <code>true</code> called every frame	call <code>super.preUpdate</code> ; use <code>play(key, true)</code>
The hero falls through the level	no collider with the ground layer, or <code>setCollisionByProperty</code> missing, or the property not set on the tiles in Tiled	check all three; <code>debug: true</code> shows the bodies
The hero cannot jump from a platform	the platform is an <code>overlap</code> , or a group made with <code>this.add.group()</code> without bodies - so <code>blocked.down</code> is never set	use <code>this.physics.add.collider</code>

What students see	Cause	Fix
Tilemap has no tileset "platform". Its tilesets are ... (with the list of names) in the console, then (in these projects) Error: The map has no tileset called "platform"	the first argument of <code>addTilessetImage</code> must be the tileset's name <i>inside the map</i>, not the image key	use the name shown in Tiled's Tilesets panel
Invalid Tilemap Layer ID: ground then Valid tilelayer names: ... in the console	a layer name typed with the wrong case	layer names are case-sensitive and must match Tiled exactly
TS2322 [ERROR]: Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.	<code>createLayer</code> can return either kind of layer	as <code>Phaser.Tilemaps.TilemapLayer</code>, with a comment (the projects' <code>createLayer</code> helper)
TS2345 [ERROR]: Argument of type 'Body StaticBody Tile GameObjectWithBody' is not assignable to parameter of type 'Sprite'.	passing a collider/overlap callback's argument straight to a method	cast it: <code>coin as Phaser.Physics.Arcade.Sprite</code> (Chapter 8)
Touching the air next to spikes kills the hero, or the hero dies as soon as the level starts	overlap with a tilemap layer calls back for empty tiles too	check <code>tile.properties.hazard</code> (or <code>tile.index !== -1</code>) in a process callback
Every touch of a slime squashes it - even walking into it	the stomp test has no "falling" check, or no margin	<code>velocity.y > 0</code> and <code>bottom <= top + margin</code>

What students see	Cause	Fix
The hero is hurt twice by one slime, or by a slime that was just squashed	the enemy's body is still enabled during its squash tween	<code>body.enable = false</code> at the start of <code>squash()</code>
The hero slides off a moving platform	the platform is moved by a tween (or by setting <code>x</code>)	move it by velocity, as <code>MovingPlatform</code> does
Static objects (the springboard in challenge 2) are hit in the wrong place after <code>setOrigin</code> or <code>setScale</code>	static bodies do not follow changes to their game object	call <code>refreshBody()</code>

Suggested demos and live-coding

- **Build the jump live** in the simple version: start with no jump; add `setVelocityY(-JUMP_SPEED)` on SPACE with no floor check (infinite flying - students love it); add the `blocked.down` check; then print `JUMP_SPEED2 / (2 x 900)` and measure the height with `debug: true`
- **Feel, side by side.** Run the simple and intermediate versions next to each other. Then in the intermediate version set `COYOTE_TIME` and `JUMP_BUFFER` to 0 and ask students to run off ledges and jump just before landing. Put them back. Most will not believe how much difference 100 ms makes until they try it
- **The empty-tile surprise.** Replace the hazard process callback with `undefined`: the hero dies the moment it goes near anything. Ask why before explaining
- **Tween vs velocity platform.** In `MovingPlatform`, swap the velocity code for a yoyo tween and `body.setDirectControl(true)`, and throttle the CPU in the browser's dev tools: the hero slides on the tween platform, and not on the velocity one
- **A new level in five minutes.** Open `tiled/level1.tmx` in Tiled, add a row of spikes and a coin, save, copy to `public/assets/maps/`, rebuild and play. Nothing in the code changes

Discussion questions

- Why does the hero need a smaller body than its picture? What would feel unfair if it were bigger? If it were much smaller?
- Coyote time lets the player jump when the hero is not on the ground. Is that cheating? Who is it for? (It makes the game do what the player *meant*.)
- The slime checks for edges by looking at the tilemap; the hero asks a function for ladders. Which design is better? What would you have to change to put a slime on a moving platform?
- `heroMeetsEnemy` lives in the scene, not in `Hero` or `Enemy`. Why? Where else could it go?
- Levels are data. What else in this game could be data rather than code? (Enemy speeds, medal times, the hero's constants - then a "moon level" with low gravity is data too.)

Extension ideas

- a level editor mode: click to place tiles and objects, then print the level as JSON
- slopes (Arcade Physics has none - discuss why Matter.js would handle them, Chapter 9)
- a crumbling platform that falls a moment after the hero lands on it
- parallax with several layers moving at different speeds, or a `tileSprite` background
- a speed-run ghost: record the hero's positions each frame and replay them as a translucent sprite on the next attempt

Assessment ideas

- **Practical:** "Add a fourth level" - a map (in Tiled) and one line in `levels.ts`. Checks Tiled skills, tile properties, object types and the data-driven design, with very little code
- **Practical:** add a new hazard tile (water that slows the hero instead of hurting it), using a new tile property
- **Code reading:** show `jump()` from the intermediate hero with the two `-1000` lines deleted; ask what the player would see and why
- **Short answer:** explain the difference between `collider` and `overlap`, and give one use of each from this chapter; explain what a process callback is for, with an example

from this chapter

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change to the code is marked with a `// CHALLENGE n` comment. Two solutions (4 and 6) also change the Tiled maps; JSON cannot hold comments, so those changes are described below.

1. Double jump

Project: `solutions/ch15_challenge_1_double_jump` (from `ch15_platformer_simple`)

Goal: a counter that is used up in the air and refilled on landing - the first taste of the hero remembering something between frames.

`src/objects/Hero.ts`

```
const AIR_JUMP_SPEED = 420; // CHALLENGE 1: the jump in the air is a little weaker
const AIR_JUMPS = 1; // CHALLENGE 1: how many extra jumps the hero gets in the air
...
// CHALLENGE 1: standing on something gives the air jumps back
if (this.body.blocked.down) {
  this.airJumpsLeft = AIR_JUMPS;
}

if (jumpPressed && this.body.blocked.down) {
  this.setVelocityY(-JUMP_SPEED); // negative y is UP
  this.scene.sound.play(JUMP_SOUND);
} else if (jumpPressed && this.airJumpsLeft > 0) {
  // CHALLENGE 1: a second jump, in the air - it REPLACES the speed, so it works the same
  // whether the hero is still rising or already falling
  this.airJumpsLeft = this.airJumpsLeft - 1;
  this.setVelocityY(-AIR_JUMP_SPEED);
  this.scene.sound.play(JUMP_SOUND, { rate: 1.3 }); // a higher-pitched jump sound
}
```

Tested with the harness, logging the vertical speed every frame while pressing jump three times: -520 (the ground jump), later -420 (the air jump), and nothing for the third press; the counter is back to 1 after landing.

Look for:

- the refill happens on the *floor*, not in the jump - otherwise the first jump would use the air jump up too
 - **setting** the speed rather than **adding** to it. `velocity.y - 420` while falling fast gives a feeble second jump; while rising it gives a rocket. Setting it makes the second jump the same every time
 - `AIR_JUMPS` as a constant: a triple jump is then a one-character change. Some students use a boolean `hasDoubleJumped` ; that is fine for exactly one extra jump
 - walking off a ledge also allows one air jump here. Either answer is fine if the student can say which they chose
-
-

2. Springboard

Project: `solutions/ch15_challenge_2_springboard` (from `ch15_platformer_simple`)

Goal: a texture drawn in code, a static body with a collider callback, and the hero being told to move rather than moving itself.

`src/scenes/GameScene.ts`

```

if (!this.textures.exists(SPRING_KEY)) {
    // make.graphics(..., false) draws off-screen: the Graphics is only used
    // to make a texture
    const g = this.make.graphics({}, false);
    ...
    g.generateTexture(SPRING_KEY, SPRING_WIDTH, SPRING_HEIGHT);
    g.destroy();
}

// origin (0.5, 1): (x, y) is the bottom middle - here, the top of the
// ground strip
const spring = this.physics.add.staticImage(SPRING_X, 536,
SPRING_KEY).setOrigin(0.5, 1);
spring.refreshBody(); // a static body does not follow changes to the image
// by itself

this.physics.add.collider(this.hero, spring, () => {
    // only when landing on TOP of it, not walking into its side
    if (this.hero.body.touching.down) {
        this.hero.launch(SPRING_SPEED);
        this.sound.play(JUMP_SOUND, { rate: 0.7 }); // a deeper "boing"
        this.tweens.add({ targets: spring, scaleY: 0.5, duration: 80, yoyo: true
});
    }
});

```

src/objects/Hero.ts

```

// CHALLENGE 2: thrown upwards by a springboard - a jump the hero did not
// choose
public launch(speed: number): void {
    this.setVelocityY(-speed);
}

```

The speed comes from the jump-height formula in the chapter: to rise about 430 pixels (from the ground at 536 to above the top platform at 144) with gravity 900 needs `sqrt(2 x 900 x 430)`, about 880. The spring sits at `x = 570`, in the gap between two platforms, so the hero is not stopped by a platform on the way up. Tested with the harness: dropped on the spring, the hero's feet rose to `y = 93`, above the top platform; steering left in the air it landed on the top platform and collected its coins.

Look for:

- `this.textures.exists` before generating: R restarts the scene, and the texture already exists the second time
 - `refreshBody()` after `setOrigin` - without it the static body stays where the image was, and the hero bounces off thin air. A classic, and worth showing with `debug: true`
 - `touching.down` (the hero's bottom touched something) so walking into the side does not launch. Because the spring is only 20 pixels high, Arcade often lifts a hero walking into it on top of it - which then launches it. That is how most games behave; accept it
 - a `launch()` method on `Hero` (ask, don't reach in - Chapter 2) is better than the scene setting the hero's velocity itself, though both work
 - a `processCallback` returning `touching.down` is a good alternative
-
-

3. Coinrecord

Project: `solutions/ch15_challenge_3_coin_record` (from `ch15_platformer_advanced`)

Goal: structured data in `localStorage`, checked on the way back in, and one class that owns it.

`src/Progress.ts`

```

// CHALLENGE 3: keep this result if it beats the record; true if it did
public static saveCoins(index: number, coins: number, total: number):
boolean {
    const records = Progress.loadCoinRecords();
    const old = records[index];
    if (old !== null && old !== undefined && old.coins >= coins) {
        return false;
    }
    records[index] = { coins, total };
    try {
        localStorage.setItem(COINS_KEY, JSON.stringify(records));
    } catch {
        // storage switched off: the record is simply not kept
    }
    return true;
}

// CHALLENGE 3: an array with a record (or null) for each level. Everything
read back from
// storage is checked: it might be missing, or not what we wrote
private static loadCoinRecords(): (CoinRecord | null)[] {
    try {
        const parsed: unknown = JSON.parse(localStorage.getItem(COINS_KEY) ?? "
[]");
        if (!Array.isArray(parsed)) {
            return [];
        }
        return parsed.map((item) => Progress.isCoinRecord(item) ? item : null);
    } catch {
        return []; // not valid JSON, or no storage
    }
}

```

src/scenes/MenuScene.ts

```

// CHALLENGE 3: a finished level shows its coin record
const record = Progress.coinRecord(index);
const coins = record === null ? "" : `    ${record.coins}/${record.total}`;
const label = open ? `${index + 1}    ${level.name}${coins}` : `${index + 1}
locked`;

```

`ResultScene` calls `Progress.saveCoins(r.level, r.coins, r.totalCoins)` when a level is completed and shows "New coin record!" if it returns `true`. The total is saved with the record because the menu has not loaded any map and cannot count the coins itself. Tested with the harness: a result of 5 coins showed "New coin record!", a later result of 3 did not, the menu showed "1 Green Hills 5/21", and a corrupted value (`{bad json}`) in storage gave a menu with no records rather than a crash.

Look for:

- the record saved only for a *completed* level (a game over should not count - the challenge says "finished")
 - `unknown` and a type guard (`item is CoinRecord`) for what comes back from `JSON.parse`, not `as CoinRecord[]` - the Chapter 6 lesson
 - `try / catch` around `JSON.parse` : it throws on bad JSON
 - the record stored by level *index* - fine while levels are never reordered; the level `key` would be more robust, and a good discussion point
-
-

4. Wall jump

Project: `solutions/ch15_challenge_4_wall_jump` (from `ch15_platformer_intermediate`)

Goal: reading `blocked.left / right` in the air, and briefly taking control away from the player - a real game-feel technique.

`src/objects/Hero.ts`

```

// CHALLENGE 4: in the air, pressed against a wall: slide down it slowly,
and jump off it
private wallJump(time: number): void {
  if (this.body.blocked.down) {
    return;
  }
  const onLeftWall = this.body.blocked.left && this.cursors.left.isDown;
  const onRightWall = this.body.blocked.right && this.cursors.right.isDown;
  if (!onLeftWall && !onRightWall) {
    return;
  }

  // sliding: never fall faster than WALL_SLIDE_SPEED while holding on
  if (this.body.velocity.y > WALL_SLIDE_SPEED) {
    this.setVelocityY(WALL_SLIDE_SPEED);
  }

  // jump pressed (just now, or a moment ago - the same buffer as a normal
  jump)
  if (time - this.jumpPressedAt <= JUMP_BUFFER) {
    this.jumpPressedAt = -1000;
    this.wallJumpDirection = onLeftWall ? 1 : -1; // away from the wall
    this.setVelocity(this.wallJumpDirection * WALL_JUMP_ACROSS, -
WALL_JUMP_UP);
    this.keysIgnoredUntil = time + WALL_JUMP_LOCK;
    this.setFlipX(this.wallJumpDirection < 0);
    this.scene.sound.play(JUMP_SOUND);
  }
}

```

src/objects/Hero.ts

```

// CHALLENGE 4: straight after a wall jump, keep going away from the wall
whatever the keys
// say - otherwise a player still holding the arrow towards the wall pulls
the hero back
if (time < this.keysIgnoredUntil) {
  this.setAccelerationX(this.wallJumpDirection * RUN_ACCELERATION);
  return;
}

```

The map. Two stone columns (tile 5, so `gid 6`) were added to the Ground layer of both copies of `level.tmj`, at columns 5 and 8 from row 4 down to row 13, making a shaft two tiles wide just right of the start. Rows 14 and 15 are left open so the hero can walk in underneath. Four coins (points of type `coin`) sit at the top of the shaft, at columns 6 and 7, rows 4 and 5: out of reach of a normal jump, which reaches about row 10. In Tiled: select the Ground layer, stamp the stone tile, then add the points on the Objects layer; save, and copy the file to `public/assets/maps/`.

Tested with the harness, with a small script in the page standing in for the player (hold towards a wall; when `blocked` against it in the air, press jump and switch direction): four wall jumps reached the top of the shaft and collected all four coins.

Look for:

- "pressing into the wall" in the test. Without it the hero sticks to walls it merely brushes
- the key lock. Without it, the player is still holding the arrow towards the wall when the jump happens, acceleration pulls the hero straight back, and it climbs a single wall - which feels like a bug. Many students discover this the hard way; it is the most useful thing in the challenge
- re-using `jumpPressedAt` gives wall jumps the same buffering as normal ones for free
- the max velocity (220) caps the push away from the wall, so `WALL_JUMP_ACROSS` above 220 does nothing - a good question to ask a student who set it to 400

5. Medal times

Project: `solutions/ch15_challenge_5_medal_times` (from `ch15_platformer_advanced`)

Goal: extend the level data, keep a clock in the HUD without flooding the registry with events, and compare "better" with a ranking.

`src/config/levels.ts`

```
// CHALLENGE 5: the times, in seconds, to beat for each medal
export interface MedalTimes {
  gold: number;
  silver: number;
  bronze: number;
}
```

src/config/medals.ts

```
export type Medal = "gold" | "silver" | "bronze" | "none";

// better medals have bigger numbers, so two medals can be compared with >
// and <
export const MEDAL_RANK: Record<Medal, number> = { none: 0, bronze: 1,
silver: 2, gold: 3 };
```

src/scenes/HudScene.ts

```
// CHALLENGE 5: the clock changes every frame, so it is worked out here
// rather than sent as an
// event. Every scene's clock is the game's clock, so this.time.now here is
// the same as in
// GameScene
override update(): void {
  const start: number = this.registry.get(START_TIME);
  const end: number = this.registry.get(END_TIME);
  const now = end > 0 ? end : this.time.now;
  this.clockText.setText(((now - start) / 1000).toFixed(1));
}
```

`GameScene` puts `START_TIME` in the registry when the level starts and sets `END_TIME` in `endLevel()`, so the clock stops the moment the flag is touched (the HUD would otherwise tick on through the fade-out). `ResultScene` works out the medal with `medalFor(r.seconds, times)`, shows it in its colour with the three target times, and saves it with `Progress.saveMedal()`, which keeps only a better rank. The menu draws a coloured disc with the medal's initial beside each level.

Tested with the harness: a 5-second run of Green Hills (after teleporting to the flag) showed "GOLD medal - a new best!" and a G disc on the menu; a later result of 50 seconds showed "BRONZE medal" without "a new best", and storage still held `["gold"]`.

Look for:

- the times in `LevelInfo`, not in the result scene: data about a level lives with the level
 - no `changedata` event every frame for the clock - the HUD reading two numbers in its own `update()` is the right tool here, and a good contrast with the event-driven coins and lives
 - `MEDAL_RANK` (or an array index) to compare medals; string comparison (`"gold" > "bronze"`) happens to work alphabetically for some pairs and not others
 - the medal times are guesses; ask how they would set them properly (play-testing: gold about the designer's best, bronze easy)
-
-

6. A new enemy

Project: `solutions/ch15_challenge_6_new_enemy` (from `ch15_platformer_advanced`)

Goal: add a class to an existing design through its interface, and change the interface when the design needs it.

`src/objects/Enemy.ts`

```
export interface Enemy {
  body: Phaser.Physics.Arcade.Body;
  isSquashed(): boolean;
  squash(): void;
  canBeStomped(): boolean; // CHALLENGE 6: false for an enemy that hurts
  even from above
}
```

`src/objects/Ghost.ts`

```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);

    const distance = Phaser.Math.Distance.BetweenPoints(this, this.target);
    if (distance < NOTICE_DISTANCE) {
        // aim at the hero's middle, not its feet (its origin is at the feet)
        this.scene.physics.moveTo(this, this.target.x, this.target.y - 24,
SPEED);
    } else {
        this.setVelocity(0, 0);
    }
    this.setFlipX(this.body.velocity.x > 0);
}
```

src/scenes/GameScene.ts

```
// CHALLENGE 6: only an enemy that CAN be stomped is squashed; the rest hurt
from any side
if (falling && feetOnTop && enemy.canBeStomped()) {
    enemy.squash();
}
```

Adding `canBeStomped()` to the interface makes the compiler list every class that must now have it: `Slime` and `Bat` each gain a three-line method returning `true`. `ghost.png` was copied from the asset library into `public/assets/spritesheets/`, loaded in `PreloadScene`, and given an animation. Ghosts are made from points of type `ghost`, with no collider (they drift through walls), and join the one enemy overlap.

The maps. Points of type `ghost` were added to the Objects layer of both copies of each map: level 1 at column 62, row 8; level 2 at column 25, row 13; level 3 at column 45, row 8 and column 88, row 7.

Tested with the harness: a ghost 150 pixels away drifted towards the hero; dropping the hero on top of the ghost cost a life, and dropping it on a slime still squashed the slime.

Look for:

- the interface changed, rather than `if (enemy instanceof Ghost)` in the scene. The `instanceof` answer works, but the scene then has to know every kind of enemy - exactly what the interface was there to avoid. A great discussion
- `Ghost.squash()` is an empty method the scene never calls. Ask whether that is a smell: splitting the interface (`Enemy` and `Stompable` extends `Enemy`) is the cleaner design,

and some students will find it

- a slow speed - a ghost as fast as the hero cannot be escaped
- `moveTo` every frame steers the ghost continuously; compare the bat, which aims once and misses

Slides

Chapter 15

Platformer

Run, jump, and do not fall down the hole



1 / 20

Today

- gravity, `body.blocked.down` and a jump
- **game feel**: acceleration, a variable jump, coyote time, jump buffering
- a level from **Tiled** whose tiles say what they are
- enemies to stomp, hazards that are fair
- one-way and moving platforms, ladders
- lives, checkpoints, a HUD and a level select

2 / 20

Gravity

```
physics: {  
  default: "arcade",  
  arcade: {  
    gravity: { x: 0, y: 900 },  
    debug: false, // true draws every body's outline - try it  
  },  
},
```

Every dynamic body falls - unless something holds it up

3 / 20

The hero

```
export class Hero extends Phaser.Physics.Arcade.Sprite {  
  declare body: Phaser.Physics.Arcade.Body;  
  
  constructor(scene: Phaser.Scene, x: number, y: number) {  
    super(scene, x, y, HERO_KEY, 0);  
    scene.add.existing(this); // draw it, and call its preUpdate() every frame  
    scene.physics.add.existing(this); // give it a dynamic Arcade Physics body  
  }  
}
```

- `declare body` - narrows `Body | StaticBody | null` to the body it really has
- the body is **smaller than the picture** - fairer

4 / 20

Standing, and jumping

```
if (jumpPressed && this.body.blocked.down) {  
    this.setVelocityY(-JUMP_SPEED); // negative y is UP  
    this.scene.sound.play(JUMP_SOUND);  
}
```

- `blocked.down` - a collider stopped the body moving down: **standing on something**
- a jump is a velocity; gravity bends it into an arc

5 / 20

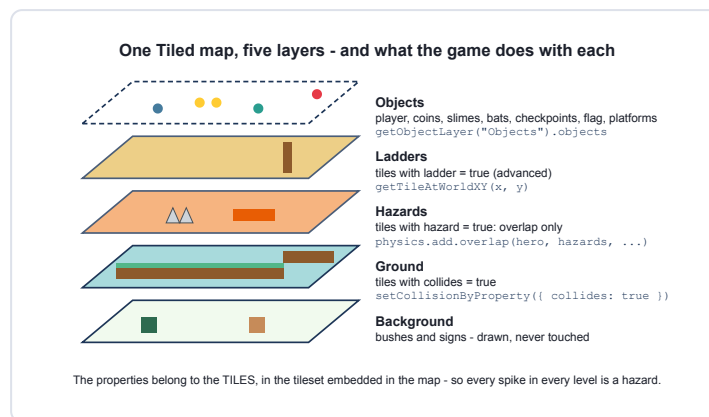
Try it: the simple platformer

- play `ch15_platformer_simple`
- how does the jump *feel*?
- set `debug: true` in `main.ts` and look at the bodies



6 / 20

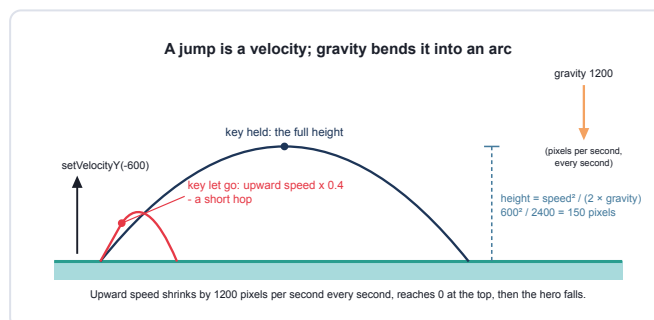
A level from Tiled



```
ground.setCollisionByProperty({ collides: true });
```

7 / 20

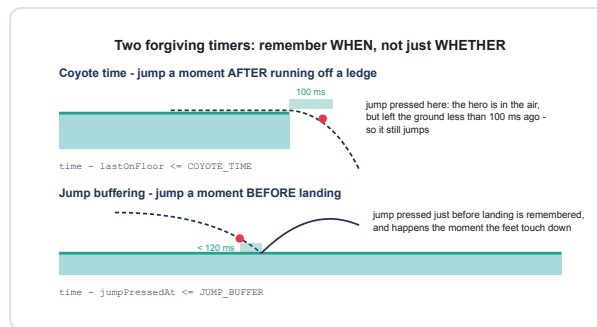
Feel 1: a jump you control



```
if (released && this.body.velocity.y < 0) {  
    this.setVelocityY(this.body.velocity.y * JUMP_CUT);  
}
```

8 / 20

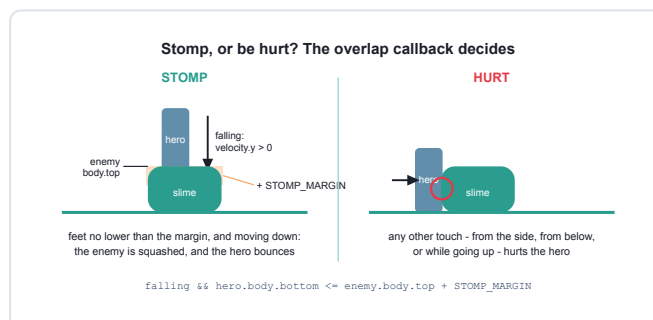
Feel 2: remember WHEN



```
const recentlyOnFloor = time - this.lastOnFloor <= COYOTE_TIME;  
const recentlyPressed = time - this.jumpPressedAt <= JUMP_BUFFER;
```

9 / 20

Stomp, or be hurt?



`squash()` switches the enemy's body off **at once**

10 / 20

Fair hazards

```
this.physics.add.overlap(this.hero, hazards, () => this.hurtHero(), (_hero, tile) => {  
    return this.isDeadly(tile as Phaser.Tilemaps.Tile);  
});  
  
private isDeadly(tile: Phaser.Tilemaps.Tile): boolean {  
    return tile.properties.hazard === true && this.hero.body.bottom > tile.pixelY + HAZARD_MARGIN;  
}
```

Overlapping a tile layer calls back for **empty tiles too**

11 / 20

Try it: the intermediate platformer

- play `ch15_platformer_intermediate`
- set `COYOTE_TIME` and `JUMP_BUFFER` to 0 - play - put them back
- open `tiled/level.tmj` in Tiled



12 / 20

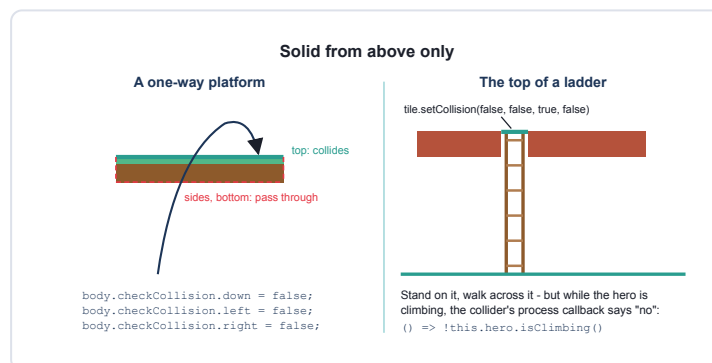
One scene, every level

```
export const LEVELS: LevelInfo[] = [
  { key: "level1", file: "assets/maps/level1.tmj", name: "Green Hills", skyTint: 0xffffffff },
  { key: "level2", file: "assets/maps/level2.tmj", name: "Deep Caves", skyTint: 0x7080a0 },
  { key: "level3", file: "assets/maps/level3.tmj", name: "Lava Castle", skyTint: 0xff9070 },
];
```

A new level = a map + one line. **Data-driven**

13 / 20

Solid from above only



Moving platforms move by **velocity**: the engine carries the rider

14 / 20

Ladders: pass a function

```
export type LadderFinder = (x: number, y: number) => number | null;  
  
this.hero = new Hero(this, start.x!, start.y!, (x, y) => this.findLadder(x, y));
```

- the hero does not know about tilemaps
- "climb" state: gravity off, arrows move up and down

15 / 20

Enemies share an interface

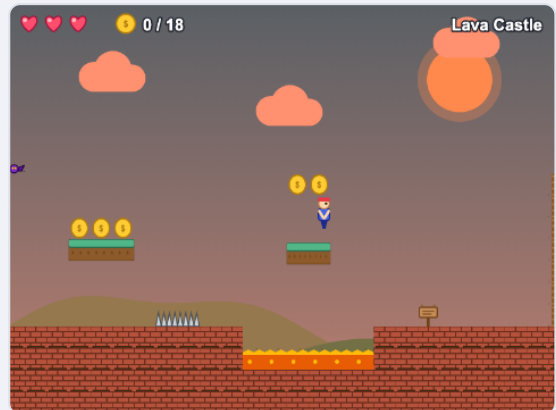
```
export interface Enemy {  
  body: Phaser.Physics.Arcade.Body;  
  isSquashed(): boolean;  
  squash(): void;  
}
```

class Bat extends Phaser.Physics.Arcade.Sprite implements Enemy - hover, swoop, return

16 / 20

Lives, checkpoints, HUD

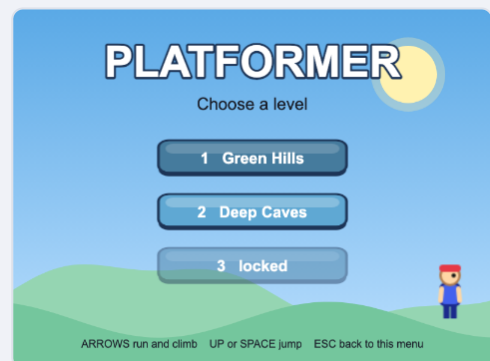
- being hurt costs a life; come back at the last **checkpoint**, flashing
- `HudScene` on top, redrawn on the registry's "changedata" event
- `Progress` keeps unlocked levels in `localStorage`



17 / 20

Try it: the advanced platformer

- play all three levels of `ch15_platformer_advanced`
- climb a ladder; ride both kinds of moving platform
- find `makeLadderTopsSolid()` - why is the collider switched off while climbing?



18 / 20

Summary

- gravity + `blocked.down` + an upward velocity = a jump
- acceleration, drag, jump cut, coyote time, buffering = **feel**
- tiles with properties; objects from the Objects layer
- stomp or hurt in the overlap callback; fair hazards in a process callback
- `checkCollision` for one-way; velocity for moving platforms
- one data-driven scene, a HUD scene, lives and checkpoints

19 / 20

Challenges

1. **Double jump** - once more, in the air
2. **Springboard** - drawn with `Graphics`, it throws the hero up
3. **Coin record** - best coins per level, on the level select
4. **Wall jump** - slide down walls, jump off them
5. **Medal times** - gold, silver, bronze, and a clock in the HUD
6. **A new enemy** - ghosts that cannot be stomped

20 / 20